

RAG Document Reader

System Architecture and Implementation Documentation

Field	Details
Status	Active development
Version	Work in progress - April 2026
Document type	Internal technical documentation
Primary stack	FastAPI backend, Next.js 14 frontend, LangGraph pipeline

1. Objective

The RAG Document Reader is a lightweight full-stack application for uploading PDF or DOCX files, indexing their extracted content, and answering user questions against the uploaded document. The system combines deterministic document ingestion, chunking, hybrid retrieval, and a grounded LLM response pipeline so users can ask natural-language questions while still seeing the source chunks used to generate each answer.

2. Current Scope

- Upload and process PDF and DOCX files through the FastAPI backend with structured validation and error handling.
- Extract text from PDFs using PyPDFLoader with automatic fallback to OCR (pdf2image + Tesseract) when text is not selectable.
- Extract embedded images from PDFs using PyMuPDF and store them as base64 for UI rendering alongside text responses.
- Read DOCX files using LangChain’s Docx2txtLoader for consistent document ingestion.
- Perform **structure-aware document parsing** by splitting content into sections based on regulatory numbering patterns (e.g., 9.2.2), preserving section IDs, titles, and hierarchy.
- Detect table-like content using heuristic rules (numeric density, column patterns, and keywords) and tag chunks for structured retrieval.
- Filter out Table of Contents (TOC) sections and dynamically expand relevant sections using TOC-based root matching for better contextual coverage.
- Apply **hybrid chunking strategy**:
 - Structure-based splitting for sections
 - Recursive chunking for large sections with overlap
 - Metadata enrichment (page, section, title, table flag, chunk type)This ensures better semantic grouping and retrieval accuracy.

3. High-Level Architecture

Layer	Implementation	Responsibility
Frontend	Next.js 14, TypeScript, React	Upload documents, submit questions, show results and retrieved chunks.
API	FastAPI	Expose health, document upload, ask, and status endpoints.
Document loaders	PyPDFLoader, Docx2txtLoader, pdf2image, pytesseract, PyMuPDF	Extract text from PDFs and DOCX files with OCR fallback; additionally extract images from PDFs and store as base64 for UI rendering.
Chunking	Hybrid (Structure-based + RecursiveCharacterTextSplitter)	Perform section-aware chunking with ~2000 -character size and overlap; maintain hierarchy and split large sections into sub-chunks.
Retrieval	FAISS index plus BM25	Merge semantic and lexical matches with

Layer	Implementation	Responsibility
		Reciprocal -rank fusion.
Table Detection	Heuristic rules (numeric density, keywords, column patterns)	Identify and tag table-like content to ensure structured retrieval and accurate reconstruction in output.
Orchestration	LangGraph StateGraph	Run retrieve and generate nodes in a deterministic sequence.
LLM	gpt-4o-mini via ChatOpenAI	Generate grounded answers from retrieved context at temperature 0.2.

4. Document Ingestion and OCR

The backend accepts only PDF and DOCX files and processes them based on file type. DOCX files are read directly using Docx2txtLoader. PDFs are first parsed using PyPDFLoader; if the extracted text is insufficient, the system automatically falls back to OCR by converting pages into images and applying pytesseract. Additionally, images embedded in PDFs are extracted using PyMuPDF and stored for UI rendering.

- Supported formats: .pdf, .docx
- PDF fallback: triggered if extracted text is less than ~500 characters
- OCR stack: pdf2image, pytesseract, Poppler, Tesseract
- Image extraction: PyMuPDF (base64 storage for frontend display)
- Temporary files are securely deleted after processing

5. Chunking Strategy

The system uses a hybrid chunking approach that combines structure-based splitting with recursive chunking. Documents are first divided into logical sections (e.g., 9.2.2), and large sections are further split using RecursiveCharacterTextSplitter. This ensures better preservation of context, hierarchy, and table integrity.

Parameter	Value	Purpose
chunking_type	Structure + Recursive	Maintains section hierarchy while handling large text efficiently
chunk_size	2000 characters	Captures full clauses, tables, and regulatory context
chunk_overlap	400 characters	Preserves continuity across chunk boundaries
metadata	page, section, title, table flag	Enables structured retrieval and context-aware responses
separators	\n\n, \n, ., :, space, empty	Gradual fallback from semantic to granular splitting

6. Hybrid Retrieval

The active backend builds a FAISS IndexFlatIP index from OpenAI embedding vectors and normalizes vectors for cosine-style inner-product search. In parallel, BM25 provides lexical matching for exact terms, regulation numbers, acronyms, and other keyword-heavy queries. Results are merged through reciprocal-rank fusion with a 0.6 semantic weight and a 0.4 BM25 weight.

Retriever	Current behavior	Why it matters
FAISS dense search	Embeds chunks with text-embedding-ada-002 and searches normalized vectors.	Finds conceptually similar passages even when wording differs.
BM25 sparse search	Lowercases and splits text, then returns the top keyword matches.	Handles exact terms, section labels, identifiers, and named entities.
Fusion	Combines ranked results using weighted reciprocal-rank scoring.	Balances semantic relevance with precise keyword evidence.

7. Retrieval-Generation Graph

The query flow is implemented as a LangGraph StateGraph. It is a stateful execution pipeline rather than a domain knowledge graph. The state contains the user question, retrieved context blocks, and final answer.

1. Retrieve node expands the user query to prioritize definitions, clauses, tables, rules, and exact phrases.
2. Hybrid retrieval returns the most relevant chunks with source, page, content, and relevance metadata.
3. Generate node assembles the retrieved context and asks gpt-4o-mini to answer only from that context.
4. The graph runs sequentially as Retrieve -> Generate -> End.

8. API Surface

Endpoint	Method	Purpose
/health	GET	Basic liveness check returning { ok: true }.
/	GET	Root metadata with endpoint list and API status.
/status	GET	Current document name, chunk count, token usage, and retrieval method.
/documents	POST	Upload and index a PDF or DOCX document.
/ask	POST	Ask a question against the currently indexed document.

9. Frontend Experience

The frontend is a Next.js 14 application that defaults to `http://localhost:8080` for API calls unless `NEXT_PUBLIC_API_BASE_URL` is provided. Users select a PDF or DOCX file, process it, ask a question, and review the answer together with retrieved source chunks. The interface also shows cumulative input and output token usage returned by the backend.

10. Runtime Configuration

Setting	Used by	Notes
OPENAI_API_KEY	Backend	Required for embeddings and chat generation.
POPLER_PATH	Backend OCR	Optional path used by pdf2image when Poppler is not on PATH.
TESSERACT_CMD	Backend OCR	Optional path to the Tesseract executable; defaults to the common Windows install path.
NEXT_PUBLIC_API_BASE_URL	Frontend	Optional backend URL override for the Next.js app.

11. Local Execution

Backend:

```
.\.venv\Scripts\activate
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8080
```

Frontend:

```
cd frontend
npm install
npm run dev
```

12. Known Implementation Notes

- The app currently stores the indexed document in process memory, so a backend restart clears the active index.
- Only one uploaded document is active at a time in the current runtime object.
- OCR requires local Poppler and Tesseract availability in addition to Python packages.
- The backend tracks LLM input/output token usage for question answering; embedding token accounting is not exposed by the active main.py response.
- Future enhancements may include persistent vector storage, multi-document workspaces, reranking, streaming answers, and authentication.